



FACULTAD DE INGENIERÍA
UNIVERSIDAD PERUANA LOS ANDES

ANÁLISIS DE RENDIMIENTO CON SQL PROFILER Y EXTENDED EVENTS

Monitorea, analiza y optimiza el rendimiento de tu base de datos de manera inteligente y profesional



¿QUÉ ES SQL PROFILER?



SQL Profiler es una herramienta de monitoreo incluida en SQL Server que captura y almacena eventos en tiempo real generados por SQL Server.

Permite rastrear actividades del servidor y analizar el comportamiento de consultas, procedimientos almacenados, transacciones, errores y más.

¿PARA QUÉ SIRVE?

- ✓ Diagnosticar problemas de rendimiento.
- ✓ Identificar consultas lentas.
- ✓ Monitorear actividad de usuarios.
- ✓ Auditoría de cambios y errores.
- ✓ Análisis detallado de eventos.



EJEMPLO APLICADO EN BASE DE DATOS: GRADOS_Y_TITULOS CON SQL PROFILER

Escenario:
Analizar el rendimiento de consultas sobre la base de datos GRADOS_Y_TITULOS.

- Pasos:**
1. Abrir SQL Profiler.
 2. Conectar al servidor.
 3. Seleccionar eventos: SQL_BatchCompleted, RPC_Completed, SP_StmCompleted, ErrorLog.
 4. Filtrar por Base de datos = GRADOS_Y_TITULOS.
 5. Ejecutar consultas y capturar eventos.
 6. Analizar duración, CPU, lecturas y plan de ejecución.

Consulta de ejemplo analizada:

```
SELECT g.AlumnoID, a.Nombres, a.Apellidos, g.FechaGrado
FROM Grados g
JOIN Alumnos a ON g.AlumnoID = a.AlumnoID
WHERE g.FechaGrado BETWEEN '2024-01-01' AND '2024-12-31';
```

Resultado esperado:

Identificar consultas lentas, detectar bloqueos, evaluar uso de índices y optimizar tiempos de respuesta.



BENEFICIOS DE SQL PROFILER

- ✓ Fácil de usar con interfaz gráfica.
- ✓ Ideal para análisis rápidos.
- ✓ Útil para diagnóstico básico.
- ✓ Permite grabar y reproducir eventos.
- ✓ Adecuado para entornos de desarrollo y pruebas.



TABLA COMPARATIVA: SQL PROFILER vs EXTENDED EVENTS

CARACTERÍSTICA	SQL PROFILER	EXTENDED EVENTS
Tecnología	Antigua (Legacy)	Moderna
Rendimiento	Mayor impacto en el sistema	Bajo impacto, alta eficiencia
Flexibilidad	Limitada	Alta y personalizable
Filtros	Básicos	Avanzados y precisos
Almacenamiento	Archivos .trc	Ring Buffer, Event File, Table
Eventos disponibles	Menos eventos	Muchos más eventos
Escalabilidad	No recomendado en producción	Recomendado para producción
Facilidad de uso	Interfaz gráfica	Requiere configuración (T-SQL)
Futuro	Obsoleto	Tecnología recomendada

BENEFICIOS DE EXTENDED EVENTS

- ✓ Alto rendimiento y bajo impacto.
- ✓ Gran capacidad de personalización.
- ✓ Captura eventos detallados y críticos.
- ✓ Almacenamiento flexible y eficiente.
- ✓ Recomendado para ambientes de producción y auditoría.



¿QUÉ ES EXTENDED EVENTS?



Extended Events es la herramienta moderna y poderosa de SQL Server para capturar eventos en tiempo real con mínimo impacto en el sistema.

Ofrece mayor flexibilidad, rendimiento y capacidad de filtrado que SQL Profiler.

¿PARA QUÉ SIRVE?

- ✓ Monitoreo de alto rendimiento.
- ✓ Diagnóstico de problemas complejos.
- ✓ Captura de eventos específicos.
- ✓ Análisis de espera y bloqueos.
- ✓ Auditoría avanzada y trazabilidad.



¿CUÁNDO USAR CUÁL?

USA SQL PROFILER CUANDO:

- Necesitas análisis rápidos y sencillos.
- Trabajas en entornos de desarrollo.
- Requieres una interfaz gráfica amigable.
- El impacto en el sistema no es crítico.
- Estás migrando o aprendiendo el monitoreo.

USA EXTENDED EVENTS CUANDO:

- Necesitas monitoreo en producción.
- Buscas bajo impacto y alto rendimiento.
- Requieres análisis detallado y avanzado.
- Necesitas filtrar eventos específicos.
- Quieres auditoría y trazabilidad avanzada.



BASE DE DATOS: GRADOS_Y_TITULOS

Tablas principales:

Alumnos
(AlumnoID, Nombres, Apellidos, DNI, FechaNacimiento, ...)
Grados
(GradoID, AlumnoID, FechaGrado, Modalidad, ...)
Titulos
(TituloID, AlumnoID, FechaTitulo, Especialidad, ...)
Usuarios
(UsuarioID, Nombres, Rol, ...)

EJEMPLO APLICADO EN BASE DE DATOS: GRADOS_Y_TITULOS CON EXTENDED EVENTS

Escenario:
Analizar el rendimiento de eventos críticos en la base de datos GRADOS_Y_TITULOS.

- Pasos:**
1. Crear una sesión de Extended Events.
 2. Seleccionar eventos: sql_statement_completed, wait_info, deadlock_graph, error_reported.
 3. Definir filtros: database_id = DB_ID('GRADOS_Y_TITULOS').
 4. Iniciar la sesión y ejecutar consultas.
 5. Revisar los eventos capturados.

Consulta de ejemplo analizada:

```
SELECT t.TituloID, a.Nombres, a.Apellidos, t.FechaTitulo
FROM Titulos t
JOIN Alumnos a ON t.AlumnoID = a.AlumnoID
WHERE t.FechaTitulo >= '2024-01-01';
```

Resultado esperado:

Capturar eventos críticos, detectar esperas, bloqueos y optimizar el rendimiento del sistema.



Consultas por segundo

125.4

+18.6% vs ayer

Tiempo promedio (ms)

245

+12.4% vs ayer

CPU (%)

68%

+8.2% vs ayer

Lecturas lógicas

1.2 M

+15.3% vs ayer

Consultas por hora



Eventos capturados



1. CAPTURA DE EVENTOS



SQL Profiler o Extended Events capturan eventos en tiempo real desde SQL Server.

2. ALMACENAMIENTO



Los eventos se almacenan en archivos, memoria o tablas para su análisis posterior.

3. ANÁLISIS



Se analizan consultas, esperas, bloqueos, errores y rendimiento del sistema.

4. DIAGNÓSTICO



Se identifican cuellos de botella, consultas lentas y problemas críticos.

5. OPTIMIZACIÓN



Se aplican mejoras en índices, consultas, configuración y recursos del servidor.

6. MONITOREO CONTINUO



Se monitorea constantemente para asegurar el rendimiento óptimo.

7. RESULTADOS



Mejor rendimiento, mayor disponibilidad y usuarios satisfechos.



FACULTAD DE INGENIERÍA
UNIVERSIDAD PERUANA LOS ANDES

ESTADÍSTICAS E ÍNDICES EN SQL SERVER

CREACIÓN, FRAGMENTACIÓN Y MANTENIMIENTO

Optimización del rendimiento mediante estadísticas actualizadas, índices eficientes y mantenimiento preventivo de bases de datos SQL Server.



¿QUÉ SON LAS ESTADÍSTICAS?

- Son objetos que almacenan información sobre la distribución de los datos en las columnas.
- Ayudan al Query Optimizer a elegir el mejor plan de ejecución.
- Almacenan histograma, densidad y número de filas.
- Mejoran la precisión de las estimaciones y el rendimiento de las consultas.
- Deben mantenerse actualizadas para que el optimizador tome decisiones correctas.



¿QUÉ SON LOS ÍNDICES?

- Estructuras que permiten acceder a los datos de manera rápida sin explorar toda la tabla.
- Funcionan como un índice de un libro.
- Reducen lecturas lógicas y físicas.
- Mejoran el rendimiento de búsquedas, filtros, ordenamientos y uniones (JOIN).
- Pueden aumentar el espacio en disco y afectar las escrituras.



COMPARACIÓN DE BÚSQUEDA

SIN ÍNDICE (ESCANEO COMPLETO)



Lee todas las filas

VS

CON ÍNDICE (B-TREE)

- ✓ Claves primarias y foráneas
- ✓ Columnas usadas en filtros
- ✓ Columnas usadas para ordenamiento
- ✓ Columnas en consultas frecuentes

¿QUÉ COLUMNAS EVITAR?

- ✗ Columnas con baja cardinalidad
- ✗ Columnas con muchos cambios
- ✗ Columnas grandes (text, imagen)
- ✗ Columnas no usadas en consultas



BENEFICIOS DE LAS ESTADÍSTICAS



¿CUÁNDO USAR CADA UNO?



TIPOS DE ÍNDICES

CLUSTERED INDEX

- Ordena físicamente los datos de la tabla.
- Solo puede existir uno por tabla.
- Ideal para claves primarias.

NONCLUSTERED INDEX

- Estructura separada de los datos.
- Puede haber varios por tabla.
- Mejora búsquedas específicas.

UNIQUE INDEX

- Garantiza valores únicos.
- Puede ser clustered o nonclustered.
- Útil para claves alternas.

COMPOSITE INDEX

- Índice sobre varias columnas.
- El orden de las columnas es importante.
- Ideal para consultas con múltiples filtros.

FILTERED INDEX

- Índice sobre un subconjunto de filas.
- Reduce tamaño y mejora rendimiento.
- Basado en una condición (filtro).

¿QUÉ ES LA FRAGMENTACIÓN?

- Ocurre cuando las páginas del índice no están contiguas.
- Aumenta las lecturas de disco.
- Reduce el rendimiento de las consultas.
- Existen dos tipos: interna y externa.
- Requiere mantenimiento periódico.



MANTENIMIENTO DE ÍNDICES

REORGANIZE

- Reacomoda los datos sin reconstruir.
- Mantiene el índice en línea.
- Recomendado cuando la fragmentación está entre 5% y 30%.

REBUILD

- Reconstruye el índice desde cero.
- Elimina completamente la fragmentación.
- Recomendado cuando la fragmentación es mayor a 30%.

ESTADÍSTICAS DE LOS ÍNDICES



BASE DE DATOS: GRADOS_Y_TITULOS

ASPECTO	ESTADÍSTICAS	ÍNDICES
Función	Proveen información sobre la distribución de datos.	Estructuras que aceleran el acceso a los datos.
Rendimiento	Mejora la calidad del plan de ejecución.	Reduce el tiempo de búsqueda y lecturas.
Espacio ocupado	Muy bajo (KB).	Alto (MB/GB según tamaño y cantidad).
Mantenimiento	Actualización periódica.	Reorganizar o reconstruir periódicamente.
Impacto	Impacto mínimo en escrituras.	Manual (REORGANIZE / REBUILD).
Actualización	Automática o manual (UPDATE STATISTICS).	Manual (REORGANIZE / REBUILD).
Optimización	Ayuda al optimizador de consultas.	Mejora el acceso físico a los datos.
Uso por Query Optimizer	Usa estadísticas para estimar cardinalidad.	Usa índices para elegir el plan más eficiente.

EJEMPLOS APLICADOS EN LA BASE DE DATOS: GRADOS_Y_TITULOS

1. CREACIÓN DE ÍNDICE
(Tabla: ALUMNOS)
Crear índice sobre DNI y Apellidos.

CREATE NONCLUSTERED INDEX IX_Alumnos_DNI_Apellidos ON ALUMNOS (DNI, Apellidos);

Mejora búsquedas por DNI y Apellidos en reportes.

2. CREACIÓN DE ÍNDICE
(Tabla: TITULOS)
Crear índice sobre FechaTitulo.

CREATE NONCLUSTERED INDEX IX_Titulos_FechaTitulo ON TITULOS (FechaTitulo);

Optimiza consultas por rangos de fechas de titulación.

3. ACTUALIZAR ESTADÍSTICAS
(Tabla: GRADOS)
Actualizar estadísticas manualmente.

UPDATE STATISTICS GRADOS WITH FULLSCAN;

Útil después de cargas masivas o cambios importantes.

4. FRAGMENTACIÓN
(Tabla: TITULOS)
Índice IX_Titulos_FechaTitulo fragmentado al 45%.

Recomendación:
ALTER INDEX IX_Titulos_FechaTitulo ON TITULOS REBUILD;

Rebuild recomendado (porcentaje > 30%).

5. CONSULTA OPTIMIZADA
(UNIÓN TABLAS)
Consulta entre ALUMNOS, GRADOS y TITULOS.

SELECT a.Apellidos, a.Nombres, g.NombreGrado, t.FechaTitulo FROM ALUMNOS a JOIN GRADOS g ON a.IdAlumno = g.IdAlumno JOIN TITULOS t ON g.IdGrado = t.IdGrado WHERE t.FechaTitulo >= '2024-01-01';

Índices + estadísticas actualizadas = consultas más rápidas y eficientes.





FACULTAD DE INGENIERÍA
UNIVERSIDAD PERUANA LOS ANDES

ADMINISTRACIÓN DE TRANSACCIONES Y BLOQUEOS

Control, Consistencia y Concurrency en SQL Server

Las transacciones garantizan la integridad de los datos. Los bloqueos controlan el acceso concurrente para evitar conflictos y asegurar la consistencia de la base de datos.



¿QUÉ ES UNA TRANSACCIÓN?

- Es un conjunto de operaciones que se ejecutan como una unidad lógica.
- Cumple con las 4 propiedades ACID:
- ✓ Atomicidad: todo o nada.
- ✓ Consistencia: mantiene la integridad.
- ✓ Aislamiento: operaciones independientes.
- ✓ Durabilidad: los cambios persisten.
- Ejemplos: inserciones, actualizaciones, eliminaciones.



TRANSACCIONES EN SQL SERVER

- SQL Server usa BEGIN TRAN, COMMIT y ROLLBACK para controlar transacciones.
- Permite asegurar integridad ante errores.
- En caso de fallo, se puede deshacer todos los cambios.



PROPIEDADES ACID

- Atomicidad**
Todo o nada.
- Consistencia**
Mantiene reglas de negocio.
- Aislamiento**
Operaciones independientes.
- Durabilidad**
Los cambios son permanentes.

BASE DE DATOS: GRADOS_Y_TITULOS



¿QUÉ SON LOS BLOQUEOS?

- Son mecanismos que controlan el acceso concurrente a los datos.
- Evitan que múltiples usuarios modifiquen los mismos datos al mismo tiempo.
- SQL Server aplica bloqueos automáticamente.
- Existen diferentes tipos y niveles de bloqueo.



TIPOS DE BLOQUEOS

- Shared (S)**
Permite lectura.
- Exclusive (X)**
Permite escritura.
- Update (U)**
Lectura con intención de actualización.
- Intent (IS, IX, SIX)**
Indican intención de bloqueo.
- Schema (Sch-S, Sch-M)**
Protegen la estructura de los objetos.

NIVELES DE BLOQUEO

NIVEL	DESCRIPCIÓN	ALCANCE
Row (Fila)	Bloquea una fila específica.	Fila
Page (Página)	Bloquea una página (8 KB).	Página
Table (Tabla)	Bloquea toda la tabla.	Tabla
Database (Base de datos)	Bloquea toda la base de datos.	Base de datos

MATRIZ DE COMPATIBILIDAD DE BLOQUEOS

	S	U	X	IS	SIX	Sch-S	Sch-M
S	✓	✓	✓	✓	✓	✓	✓
U	✓	✓	✓	✓	✓	✓	✓
X	✓	✓	✓	✓	✓	✓	✓
IS	✓	✓	✓	✓	✓	✓	✓
IX	✓	✓	✓	✓	✓	✓	✓
SIX	✓	✓	✓	✓	✓	✓	✓
Sch-S	✓	✓	✓	✓	✓	✓	✓
Sch-M	✓	✓	✓	✓	✓	✓	✓

✓ Compatible X Incompatible

EJEMPLO DE TRANSACCIÓN EN GRADOS_Y_TITULOS

EJEMPLO: INSERCIÓN DE TÍTULO

```
BEGIN TRAN;  
INSERT INTO TITULOS  
(IdAlumno, FechaTitulo, Especialidad)  
VALUES (101, '2023-05-20', 'Ingeniería de Sistemas');  
  
INSERT INTO DETALLE_TITULO  
(IdTitulo, Observaciones)  
VALUES (SCOPE_IDENTITY(), 'Titulación por tesis');  
  
COMMIT TRAN;
```

EJEMPLO CON ERROR (ROLLBACK)

```
BEGIN TRAN;  
INSERT INTO TITULOS  
(IdAlumno, FechaTitulo, Especialidad)  
VALUES (101, '2023-05-20', 'Ingeniería de Sistemas');  
  
-- Error simulando una falla  
INSERT INTO DETALLE_TITULO (IdTitulo, Observaciones)  
VALUES (9999, 'Data inválida');  
  
ROLLBACK TRAN;
```

VERIFICACIÓN

```
SELECT * FROM TITULOS WHERE IdAlumno = 101;  
-- No se insertó nada  
  
SELECT * FROM DETALLE_TITULO  
WHERE IdTitulo = 9999;  
-- No se insertó nada
```



NIVELES DE AISLAMIENTO

NIVEL	DESCRIPCIÓN
READ UNCOMMITTED	Permite lecturas sucias.
READ COMMITTED	Evita lecturas sucias.
REPEATABLE READ	Evita lecturas no repetibles.
SERIALIZABLE	Máximo nivel de aislamiento.
SNAPSHOT	Usa versiones para consistentes.

PROBLEMAS DE CONCURRENCIA

- Lecturas sucias**
Se leen datos no confirmados.
- Lecturas no repetibles**
Los datos cambian entre lecturas.
- Fantasmas (Phantom Reads)**
Aparecen filas nuevas en la consulta.
- Bloqueos (Blocking)**
Las operaciones esperan a otras.
- Interbloqueos (Deadlocks)**
Dos sesiones se bloquean entre sí.

DEADLOCK (INTERBLOQUEO)

Ocurre cuando dos o más transacciones se bloquean mutuamente.



SQL Server detecta y resuelve el deadlock cancelando una de las transacciones.

MONITOREO DE BLOQUEOS

CONSULTA PARA VER BLOQUEOS ACTIVOS

```
SELECT  
request_session_id AS Sesión_Bloqueada,  
resource_type AS Tipo_Recurso,  
resource_description AS Recurso,  
request_mode AS Modo_Solicitado,  
request_status AS Estado  
FROM sys.dm_tran_locks;
```



CONSULTA PARA VER SESIONES BLOQUEADAS

```
SELECT  
blocking_session_id AS Sesión_Bloqueadora,  
session_id AS Sesión_Bloqueada,  
wait_type, wait_time, wait_resource  
FROM sys.dm_os_waiting_tasks  
WHERE blocking_session_id > 0;
```

MEJORES PRÁCTICAS

- ✓ Mantener transacciones cortas.
- ✓ Usar índices adecuados para reducir bloqueos.
- ✓ Evitar SELECT * en transacciones largas.
- ✓ Usar el nivel de aislamiento adecuado.
- ✓ Monitorear bloqueos y sesiones.
- ✓ Diseñar consultas eficientes.
- ✓ Actualizar estadísticas e índices regularmente.



COMANDOS DE CONTROL DE TRANSACCIONES

- BEGIN TRAN** — Inicia una transacción.
- COMMIT TRAN** — Confirma los cambios.
- ROLLBACK TRAN** — Deshace los cambios.
- SAVE TRAN** — Crea un punto de guardado.
- SET TRANSACTION ISOLATION LEVEL** — Define el nivel de aislamiento.



EJEMPLOS APLICADOS EN GRADOS_Y_TITULOS

- TRANSACCIÓN PARA REGISTRAR UN GRADO Y TÍTULO**
Se registra el grado y posteriormente el título del alumno de forma atómica.
- BLOQUEO POR ACTUALIZACIÓN SIMULTÁNEA**
Dos usuarios intentan actualizar el mismo registro de ALUMNOS.
- SOLUCIÓN CON NIVEL DE AISLAMIENTO**
Usando READ COMMITTED SNAPSHOT se reducen bloqueos.
- DETECCIÓN Y RESOLUCIÓN DE DEADLOCK**
SQL Server cancela automáticamente una transacción para liberar recursos.

DASHBOARD EJECUTIVO DE TRANSACCIONES Y BLOQUEOS



FLUJO DE ADMINISTRACIÓN DE TRANSACCIONES Y BLOQUEOS





FACULTAD DE INGENIERÍA
UNIVERSIDAD PERUANA LOS ANDES

ANÁLISIS DE PLANES DE EJECUCIÓN EN SQL SERVER

Comprende, interpreta y optimiza el rendimiento de tus consultas para tomar mejores decisiones.



¿QUÉ ES UN PLAN DE EJECUCIÓN?

- Es una representación gráfica o en texto que muestra cómo SQL Server ejecuta una consulta.
- El plan muestra las operaciones, el orden de ejecución y los recursos utilizados.
- Permite identificar cuellos de botella y oportunidades de optimización.



¿PARA QUÉ SIRVE?

- ✓ Comprender el comportamiento de las consultas.
- ✓ Identificar operaciones costosas.
- ✓ Detectar problemas de rendimiento.
- ✓ Optimizar índices y estructura.
- ✓ Reducir tiempos de ejecución.



TIPOS DE PLANES DE EJECUCIÓN

PLAN ESTIMADO

- Generado por el optimizador antes de ejecutar la consulta.
- No usa estadísticas en tiempo real.
- Se obtiene con: SET SHOWPLAN_XML ON



PLAN REAL

- Generado después de ejecutar la consulta.
- Incluye tiempos reales y filas procesadas.
- Se obtiene con: SET STATISTICS XML ON o Actual Plan.



¿CÓMO OBTENER UN PLAN?

DESDE SSMS (GRÁFICO)

- Ejecutar la consulta.
- Clic en Display Estimated Execution Plan (Ctrl + L) o Include Actual Execution Plan (Ctrl + M).



DESDE SSMS (TEXTO)

- Usar SET SHOWPLAN_TEXT ON (estimado) o SET STATISTICS PROFILE ON (real).

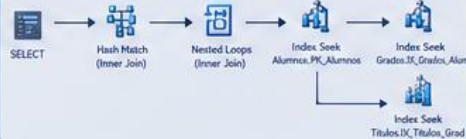


EJEMPLO EN LA BASE DE DATOS: GRADOS_Y_TITULOS

CONSULTA DE EJEMPLO

```
SELECT a.AlumnoID, a.Nombres, a.Apellidos, g.FechaGrado, t.FechaTitulo  
FROM Alumnos a  
JOIN Grados g ON a.AlumnoID = g.AlumnoID  
JOIN Titulos t ON g.GradoID = t.GradoID  
WHERE t.FechaTitulo >= '2024-01-01';
```

PLAN ESTIMADO (GRÁFICO)



ANÁLISIS

- Se usan Index Seek en las tres tablas.
- Hash Match para el primer JOIN.
- Buen uso de índices.
- Costo estimado total: 36%
- Plan eficiente.

DASHBOARD EJECUTIVO DE RENDIMIENTO

Tiempo promedio de consulta (ms)

125.4

▼ -18.2% vs ayer

CPU Promedio (%)

42%

▼ -10.5% vs ayer

Lecturas lógicas (por consulta)

1.2 M

▼ -21.6% vs ayer

Escrituras lógicas (por consulta)

245 K

▼ -15.3% vs ayer

Consultas por segundo (QPS)

320

▲ +15.8% vs ayer

Ejecuciones por estado



Top 5 Operadores por Costo



Costo total del plan

36%

Costo Relativo

Memoria Grant (MB)

128

▼ -12.4% vs ayer

Bloqueos detectados

15

▼ -6.3% vs ayer

FLUJO PARA ANALIZAR Y OPTIMIZAR PLANES DE EJECUCIÓN





FACULTAD DE INGENIERÍA
UNIVERSIDAD PERUANA LOS ANDES

OPTIMIZACIÓN DE CONSULTAS T-SQL

Escribe consultas eficientes, rápidas y escalables

Mejores prácticas, estructura, índices y análisis para obtener el máximo rendimiento en SQL Server.



¿QUÉ ES LA OPTIMIZACIÓN DE CONSULTAS?

- Es el proceso de mejorar la eficiencia de consultas T-SQL para reducir tiempo de ejecución, uso de recursos y mejorar el rendimiento general del sistema.
- Una consulta optimizada:
 - ✓ Usa menos CPU, memoria y E/S.
 - ✓ Devuelve resultados más rápido.
 - ✓ Escala mejor con grandes volúmenes de datos.
 - ✓ Reduce bloqueos y esperas.



PRINCIPIOS DE OPTIMIZACIÓN

- ✓ Devuelve solo lo necesario.
- ✓ Filtra lo antes posible.
- ✓ Usa índices adecuados.
- ✓ Evita funciones en columnas.
- ✓ Evita SELECT *.
- ✓ Usa JOINS eficientes.
- ✓ Mantén estadísticas actualizadas.



ERRORES COMUNES QUE DEBES EVITAR

- ✗ SELECT *
- ✗ Filtrados en columnas usando funciones
- ✗ Conversión de tipos implícita
- ✗ Subconsultas no correlacionadas
- ✗ JOINS innecesarios
- ✗ Uso incorrecto de OR
- ✗ Falta de índices



MEJORES PRÁCTICAS CON EJEMPLOS

PRÁCTICA	NO RECOMENDADO	RECOMENDADO
1. Evitar SELECT *	SELECT * FROM Alumnos;	SELECT IdAlumno, Nombres, Apellidos FROM Alumnos;
2. Filtrar lo antes posible	SELECT * FROM Titulos WHERE YEAR(FechaTitulo) = 2024;	SELECT * FROM Titulos WHERE FechaTitulo >= '2024-01-01' AND FechaTitulo < '2025-01-01';
3. Usar JOINS explícitos	FROM A, B WHERE A.Id = B.Id	FROM A INNER JOIN B ON A.Id = B.Id
4. Evitar funciones en columnas	WHERE UPPER(Apellidos) = 'PEREZ'	WHERE Apellidos = 'Perez' COLLATE SQL_Latin1_General_CP1_CI_AS
5. Usar EXISTS en lugar de IN	WHERE IdAlumno IN (SELECT IdAlumno FROM Grados)	WHERE EXISTS WHERE G.IdAlumno = Alumnos.IdAlumno

DASHBOARD EJECUTIVO DE OPTIMIZACIÓN



ECOSISTEMA DE OPTIMIZACIÓN DE CONSULTAS



EJEMPLO EN LA BASE DE DATOS: GRADOS_Y_TITULOS

CONSULTA ORIGINAL (LENTA)	CONSULTA OPTIMIZADA (RÁPIDA)
<pre>SELECT * FROM Alumnos A, Grados G, Titulos T WHERE A.IdAlumno = G.IdAlumno AND G.IdGrado = T.IdGrado AND YEAR(T.FechaTitulo) = 2024;</pre>	<pre>SELECT A.Nombres, A.Apellidos, G.NombreGrado, T.FechaTitulo FROM Alumnos A INNER JOIN Grados G ON A.IdAlumno = G.IdAlumno INNER JOIN Titulos T ON G.IdGrado = T.IdGrado WHERE T.FechaTitulo >= '2024-01-01' AND T.FechaTitulo < '2025-01-01';</pre>
Tiempo ejecución: 12.450 ms Lecturas lógicas: 182,345 CPU: 85% Plan: Table Scan	Tiempo ejecución: 320 ms Lecturas lógicas: 2,145 CPU: 12% Plan: Index Seek + Nested Loops

TÉCNICAS DE OPTIMIZACIÓN

- ✓ Usar índices correctamente.
- ✓ Reescribir subconsultas como JOINS.
- ✓ Usar EXISTS en lugar de IN cuando sea posible.
- ✓ Evitar funciones en condiciones WHERE.
- ✓ Aplicar TOP cuando solo se requiere un subconjunto.
- ✓ Usar CTEs o vistas de forma eficiente.
- ✓ Parametrizar consultas.



BENEFICIOS

- Consultas más rápidas.
- Menor uso de CPU y memoria.
- Reducción de bloqueos.
- Mejor escalabilidad.
- Mayor productividad del sistema.



COMPARACIÓN: CONSULTA NO OPTIMIZADA vs OPTIMIZADA

NO OPTIMIZADA

```
SELECT *
FROM Alumnos A, Grados G, Titulos T
WHERE A.IdAlumno = G.IdAlumno
AND G.IdGrado = T.IdGrado
AND YEAR(T.FechaTitulo) = 2024;
```

- ✗ Escaneo de tablas completo.
- ✗ Mayor tiempo de ejecución.
- ✗ Mayor uso de CPU y E/S.

OPTIMIZADA

```
SELECT A.Nombres, A.Apellidos, G.NombreGrado, T.FechaTitulo
FROM Alumnos A
INNER JOIN Grados G ON A.IdAlumno = G.IdAlumno
INNER JOIN Titulos T ON G.IdGrado = T.IdGrado
WHERE T.FechaTitulo >= '2024-01-01'
AND T.FechaTitulo < '2025-01-01';
```

- ✓ Uso de índices (Index Seek).
- ✓ Menor tiempo de ejecución.
- ✓ Menor uso de CPU y E/S.

HERRAMIENTAS PARA OPTIMIZACIÓN

- Plan de Ejecución (Actual Estimated).
- SET STATISTICS IO, TIME ON.
- Query Store.
- Extended Events.
- Índices y Estadísticas.



ÍNDICES: CLAVE PARA LA OPTIMIZACIÓN

Un buen índice puede reducir drásticamente el tiempo de ejecución.

- ✓ Índices Clustered
- ✓ Índices NonClustered
- ✓ Índices Compuestos
- ✓ Índices Filtrados



CHECKLIST DE OPTIMIZACIÓN

- ✓ ¿Devuelve solo las columnas necesarias?
- ✓ ¿Usa filtros eficientes?
- ✓ ¿Las condiciones permiten usar índices?
- ✓ ¿Se evitan funciones en columnas?
- ✓ ¿Los JOINS son los necesarios?
- ✓ ¿Las estadísticas están actualizadas?
- ✓ ¿Se probó el plan de ejecución?



BASE DE DATOS UTILIZADA

GRADOS_Y_TITULOS

Tablas principales:

- Alumnos (IdAlumno, Nombres, Apellidos, DNI, ...)
- Grados (IdGrado, IdAlumno, IdModalidad, ...)
- Titulos (IdTitulo, IdGrado, FechaTitulo, Especialidad, ...)

1. IDENTIFICAR

Detecta consultas lentas o costosas.



2. ANALIZAR

Revisa plan de ejecución y estadísticas.



3. DIAGNOSTICAR

Identifica cuellos de botella: scans, sorts, lookups.



4. OPTIMIZAR

Aplica técnicas, índices y reescribe la consulta.



5. VALIDAR

Prueba y compara resultados.



6. IMPLEMENTAR

Implementa cambios en producción.



7. MONITOREAR

Monitorea rendimiento y mejora continua.





FACULTAD DE INGENIERÍA
UNIVERSIDAD PERUANA LOS ANDES



CONTROL DE RECURSOS CON RESOURCE GOVERNOR

Asigna, limita y administra recursos para garantizar rendimiento y estabilidad

Resource Governor permite administrar CPU, memoria y E/S entre diferentes cargas de trabajo para evitar que una consulta consuma todos los recursos del servidor.



¿QUÉ ES RESOURCE GOVERNOR?

Es una característica de SQL Server que permite administrar y controlar el uso de recursos del servidor.

- ✓ Controla CPU, memoria y E/S.
- ✓ Aísla cargas de trabajo.
- ✓ Mejora estabilidad y rendimiento.
- ✓ Evita consultas que afecten a otros usuarios.
- ✓ Permite definir políticas y prioridades.



¿PARA QUÉ SIRVE?

- ✓ Evitar que consultas pesadas afecten al sistema.
- ✓ Garantizar rendimiento a aplicaciones críticas.
- ✓ Administrar entornos multiusuario.
- ✓ Prevenir bloqueos por consumo excesivo.
- ✓ Mejorar predictibilidad y escalabilidad.



COMPONENTES PRINCIPALES

- RESOURCE POOL**
Conjunto de recursos que se asignan a un grupo de sesiones.
- WORKLOAD GROUP**
Grupo de sesiones que comparten las mismas reglas.
- CLASSIFIER FUNCTION**
Función que clasifica las sesiones en un workload group.
- RESOURCE GOVERNOR**
Aplica y administra las reglas definidas.

CONFIGURACIÓN BÁSICA (T-SQL)

```
-- 1. Crear Resource Pools
CREATE RESOURCE POOL OLTP_POOL
WITH (MAX_CPU_PERCENT = 60, MAX_MEMORY_PERCENT = 50);

CREATE RESOURCE POOL REPORTS_POOL
WITH (MAX_CPU_PERCENT = 25, MAX_MEMORY_PERCENT = 30);

CREATE RESOURCE POOL ETL_POOL
WITH (MAX_CPU_PERCENT = 10, MAX_MEMORY_PERCENT = 15);

CREATE RESOURCE POOL ADMIN_POOL
WITH (MAX_CPU_PERCENT = 5, MAX_MEMORY_PERCENT = 5);

-- 2. Crear Workload Groups
CREATE WORKLOAD GROUP OLTP_GROUP USING OLTP_POOL;
CREATE WORKLOAD GROUP REPORTS_GROUP USING REPORTS_POOL;
CREATE WORKLOAD GROUP ETL_GROUP USING ETL_POOL;
CREATE WORKLOAD GROUP ADMIN_GROUP USING ADMIN_POOL;
```

```
-- 3. Crear Function Classifier
CREATE FUNCTION dbo.fn_RG_Classifier()
RETURNS sysname
WITH SCHEMABINDING
AS
BEGIN
    DECLARE @AppName NVARCHAR(128) = APP_NAME();
    IF @AppName = 'SistemaOLTP' RETURN 'OLTP_GROUP';
    IF @AppName = 'Reportes' RETURN 'REPORTS_GROUP';
    IF @AppName = 'ETL_Proceso' RETURN 'ETL_GROUP';
    RETURN 'ADMIN_GROUP';
END

-- 4. Habilitar Resource Governor
ALTER RESOURCE GOVERNOR ON;
WITH (CLASSIFIER_FUNCTION = dbo.fn_RG_Classifier);
ALTER RESOURCE GOVERNOR RECONFIGURE;
```

ECOSISTEMA DE RESOURCE GOVERNOR



BASE DE DATOS: GRADOS_Y_TITULOS

APLICACIONES Y CARGAS DE TRABAJO



TABLAS PRINCIPALES



ESCENARIO

El sistema académico tiene muchas consultas OLTP durante el día, pero los reportes pesados y procesos ETL se ejecutan en horarios específicos. Resource Governor asegura que ninguna carga consuma todos los recursos.

MONITOREO Y VISTAS ÚTILES

```
sys.resource_stats
-- Uso actual de recursos por pool.

sys.dm_resource_governor_resource_pools
-- Configuración de Resource Pools.

sys.dm_resource_governor_workload_groups
-- Configuración de Workload Groups.

sys.dm_exec_sessions
-- Sesiones clasificadas y su grupo asignado.

sys.dm_exec_requests
-- Solicitudes activas y uso de recursos.
```



MEJORES PRÁCTICAS

- ✓ Definir pools según criticidad del negocio.
- ✓ Asignar CPU y memoria de forma balanceada.
- ✓ Usar clasificador por APP_NAME() o LOGIN_NAME().
- ✓ Monitorear continuamente el uso de recursos.
- ✓ Evitar configuraciones muy restrictivas.
- ✓ Probar cambios en entornos controlados.



CUÁNDO USAR RESOURCE GOVERNOR

- Servidores con múltiples aplicaciones.
- Entornos con reportes pesados.
- Procesos ETL que consumen muchos recursos.
- Para garantizar rendimiento en cargas críticas.



Uso de CPU por Resource Pool



Distribución de Memoria (%)



Sesiones por Workload Group



FLUJO DE CONTROL DE RECURSOS CON RESOURCE GOVERNOR

